



Pashov Audit Group

Spicenet Security Review

April 6th 2026 - April 10th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Spicenet	4
5. Executive Summary	4
6. Findings	5
High findings	7
[H-01] Inconsistent token decimal handling in <code>update_mapping</code> function	7
[H-02] Missing recipient validation in <code>process_transaction</code> function	9
Medium findings	12
[M-01] Withdrawal confirmation burns amounts without validating against stored request	12
[M-02] Withdrawal confirmation lacks validation of <code>chain_hash</code> against stored request	15
[M-03] Failed withdrawals have no recovery path permanently locking user values	16
Low findings	18
[L-01] Hardcoded user address in <code>ExecuteIntentAuth</code> restricts chain-agnostic design	18
[L-02] Withdrawal requests persist unbounded user-controlled strings	19
[L-03] Zero-amount deposits and withdrawals are accepted	20
[L-04] Only the first call in a signed batch is executed while others are discarded	21
[L-05] Admin address is immutable no key rotation mechanism	23
[L-06] Inconsistent <code>HashSet</code> implementations across the crate	23
[L-07] Smart contract wallets unable to withdraw funds from L1 escrow	24
[L-08] Inadequate validation in <code>update_mapping</code> function for token removal	25
[L-09] Per-user withdrawal history grows unbounded with O(n) rewrite cost per request	27
[L-10] No validation of <code>log.address</code> in <code>WithdrawalProcessed</code> event enables attack	28



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Spicenet

Spicenet is a sovereign rollup built on the Sovereign SDK that implements on-chain settlement functionality for cross-chain token transfers and withdrawal intents. The settlements module handles receipt token minting, multi-chain token mapping with decimal normalization, and intent-authenticated withdrawal request processing.

5. Executive Summary

A time-boxed security review of the **pepper-research/spicenet** repository was done by Pashov Audit Group, during which **Nirlin**, **Playboieth**, **0xAlix2**, **Newspace** engaged to review **Spicenet**. A total of **15** issues were uncovered.

Protocol Summary

Project Name	Spicenet
Protocol Type	Sovereign Rollup Settlement
Timeline	April 6th 2026 - April 10th 2026

Review commit hash:

- [49342225cba3d7748599320c75c3d76515522d1b](#)
(pepper-research/spicenet)

Fixes review commit hash:

- [fd99accdce17023654c20d9de1e1e0db73ec14bb](#)
(pepper-research/spicenet)

Scope

`execute_intent_auth.rs` `process_transaction.rs` `update_mapping.rs` `lib.rs`
`error.rs` `event.rs` `get.rs` `chain_address.rs` `chain_id.rs`
`chain_scoped_data.rs` `chain_transaction.rs` `withdrawal_request.rs`
`svm_network.rs` `token_id_ord.rs` `helpers.rs` `wallet.rs`



6. Findings

Findings count

Severity	Amount
High	2
Medium	3
Low	10
Total findings	15

Summary of findings

ID	Title	Severity	Status
[H-01]	Inconsistent token decimal handling in <code>update_mapping</code> function	High	Resolved
[H-02]	Missing recipient validation in <code>process_transaction</code> function	High	Resolved
[M-01]	Withdrawal confirmation burns amounts without validating against stored request	Medium	Resolved
[M-02]	Withdrawal confirmation lacks validation of <code>chain_hash</code> against stored request	Medium	Resolved
[M-03]	Failed withdrawals have no recovery path permanently locking user values	Medium	Resolved
[L-01]	Hardcoded user address in <code>ExecuteIntentAuth</code> restricts chain-agnostic design	Low	Acknowledged
[L-02]	Withdrawal requests persist unbounded user-controlled strings	Low	Resolved
[L-03]	Zero-amount deposits and withdrawals are accepted	Low	Acknowledged
[L-04]	Only the first call in a signed batch is executed while others are discarded	Low	Resolved
[L-05]	Admin address is immutable no key rotation mechanism	Low	Resolved
[L-06]	Inconsistent <code>HashSet</code> implementations across the crate	Low	Resolved



ID	Title	Severity	Status
[L-07]	Smart contract wallets unable to withdraw funds from L1 escrow	Low	Resolved
[L-08]	Inadequate validation in <code>update_mapping</code> function for token removal	Low	Acknowledged
[L-09]	Per-user withdrawal history grows unbounded with O(n) rewrite cost per request	Low	Acknowledged
[L-10]	No validation of <code>log.address</code> in <code>WithdrawalProcessed</code> event enables attack	Low	Resolved



High findings

[H-01] Inconsistent token decimal handling in `update_mapping` function

Severity

Impact: High

Likelihood: Medium

Description

The `TokenMapConfigType` mapping (`HashMap<TokenIdOrd, HashSet<ChainAddress>>`) allows the admin to associate multiple chain-specific token addresses across different chains with a single receipt token on the rollup. When the admin calls `update_mapping`, the only validation is that the receipt token exists in `sov_bank`; there is no check that all `ChainAddress` entries mapped to a given receipt token represent tokens with the same number of decimals.

`process_deposit` (in `process_transaction.rs`) takes the raw `u128` amount directly from the decoded L1 ERC-20 `Transfer` event without decimal normalization:

```
let amount = Amount(amount_raw);
self.modify_escrow_balance(&chain_token, |b| b.checked_add(amount), state)?;
self.sov_bank.mint(Coins { amount, token_id: receipt_token }, &user, &admin, state)?;
```

Similarly, `execute_intent_auth` processes withdrawal amounts using the raw value from ABI-decoded call data.

Since the same receipt token is minted and burned regardless of which chain the deposit or withdrawal targets, amounts that represent vastly different real-world values are treated as equivalent. This is a realistic admin configuration mistake: when onboarding a new L2 where a bridge has deployed the "same" stablecoin with a different decimal representation (e.g., 18 decimals instead of the canonical 6), mapping it to the existing receipt token inadvertently introduces a cross-chain value extraction vector.

This risk is amplified for Solana (SVM) chains: the `ChainId` enum explicitly supports `Svm { network }` variants, and Solana SPL tokens commonly use 6 or 9 decimals while EVM tokens vary between 6, 8, and 18 decimals. Mapping a Solana 9-decimal token alongside an EVM 18-decimal token to the same receipt token creates a 10^9 -unit ratio arbitrage per transaction.



Attack Flow Example

1. The admin onboards USDC on a new L2 chain where the bridged USDC uses 18 decimals (this is common; several L2 bridges deploy wrapped USDC with 18 decimals). The admin maps both to the same receipt token: `receipt_USDC -> { evm:1:USDC_eth (6 decimals), evm:99:USDC_l2 (18 decimals) }`
2. Ethereum escrow holds 50,000 USDC (6 decimals) from existing depositors, i.e. `escrow[evm:1:USDC_eth] = 50_000_000_000` ($50k * 10^6$).
3. An attacker deposits 100 USDC on the L2 chain (18-decimal representation). The L1 `Transfer` event emits `value = 100_000_000_000_000_000_000` ($100 * 10^{18}$). `process_deposit` mints `100 * 10^{18}` units of `receipt_USDC` to the attacker.
4. The attacker initiates a withdrawal of `50_000_000_000 receipt_USDC` ($50k * 10^6$) targeting Ethereum:
5. `escrow[evm:1:USDC_eth]` is subtracted by `50_000_000_000` → succeeds (escrow has exactly this).
6. `50_000_000_000 receipt_USDC` transferred from attacker to admin — the attacker has `100 * 10^{18}` tokens, so this is a tiny fraction of their balance.
7. The L1 withdrawal settles on Ethereum, releasing 50,000 real USDC to the attacker.
8. **Result:** The attacker deposited \$100 worth of USDC on the L2 and extracted \$50,000 worth of USDC from the Ethereum escrow. They still hold approximately `100 * 10^{18}` receipt tokens for further extraction. The multiplier is $10^{(18 - 6)} = 10^{12}$; every 1 raw unit deposited on the 18-decimal chain can extract 10^{12} raw units from the 6-decimal chain.

Recommendation

Store the expected decimal count for each `ChainAddress` and enforce that all chain tokens mapped to the same receipt token share identical decimal counts:

```
// In update_mapping:
for (token_id, chain_tokens) in &new_mapping {
    let mut decimals_iter = chain_tokens.iter().map(|ct| ct.decimals);
    if let Some(first_decimals) = decimals_iter.next() {
        for d in decimals_iter {
            if d != first_decimals {
                return Err(SettlementsError::DecimalMismatch {
                    token_id: (*token_id).into(),
                    expected: first_decimals,
                    found: d,
                });
            }
        }
    }
}
```

Alternatively, normalize amounts during deposit/withdrawal by scaling to a canonical decimal representation. The validation-at-config-time approach is simpler and avoids precision loss edge cases.



Fix Implemented

The fix introduces a new `ChainTokenConfig` struct carrying a `decimals` field, and reworks the token mapping from `HashMap<TokenIdOrd, HashSet<ChainAddress>>` to `HashMap<TokenIdOrd, HashMap<ChainAddress, ChainTokenConfig>>` so each chain token entry now records its decimal precision. A `normalize_decimals` utility function is added that scales an amount by `10|delta|` (up or down) when the source chain token's decimals differ from the receipt token's decimals. Both the deposit path in `process_transaction` and the withdrawal path in `execute_intent_auth` now call this helper before minting or burning receipt tokens, ensuring cross-chain amounts are always converted to the correct receipt-token denomination.

[H-02] Missing recipient validation in `process_transaction` function

Severity

Impact: High

Likelihood: Medium

Description

In `process_transaction`, when processing an `ERC20::Transfer` event log, the `to` field is destructured but explicitly discarded (`to: _`). The code credits a deposit to the `from` address based solely on the `Transfer` event's `value`, without confirming that the tokens were actually sent to the rollup's escrow contract.

The `process_transaction` call path is restricted to the admin via `CallMessage::ProcessTransaction`, which limits direct exploitability. However, the absence of recipient validation removes a critical defense-in-depth layer. If the admin submits an incorrect L1 transaction proof, whether through operational error, replaying the wrong transaction hash, or key compromise, any `ERC20::Transfer` event on the target token contract will be interpreted as a valid deposit. A routine transfer between two unrelated parties (e.g., Alice transferring USDC to Bob on L1) would cause the rollup to mint a corresponding balance for Alice without any tokens entering escrow.

Because the settlement module provides no mechanism to reverse or unwind a processed deposit, an erroneously credited phantom deposit is permanent. The resulting balance is fully spendable on L2, and if the credited user withdraws, real tokens are drained from the escrow, creating a deficit that socializes the loss across all other depositors.

An additional attack vector arises with fee-on-transfer tokens (e.g., USDT when its fee mechanism is active): a single `transfer()` call emits two `Transfer` events — one for the principal amount delivered to the escrow and one for the fee routed to a fee collector. Without



`to` validation against the registered escrow address, both events are processed as deposits, minting more receipt tokens than the escrow actually received and creating an insolvency condition that compounds with every deposit.

Attack Flow Example

Scenario – Batched L1 transaction with multiple Transfer events (user-exploitable via honest admin):

1. A user crafts an L1 transaction that batches two actions atomically: (a) a legitimate deposit of 100 USDC to the escrow contract, and (b) a transfer of 50,000 USDC to a user-controlled address (e.g., via a multicall or smart contract wallet).
2. This single L1 transaction emits two `Transfer` events from the USDC contract:
3. `Transfer(User, Escrow, 100)` — the real deposit.
4. `Transfer(User, UserControlledAddr, 50,000)` — unrelated transfer in the same transaction.
5. The admin submits this L1 transaction to `process_transaction`. Since the first transfer is a legitimate escrow deposit, the admin has no reason to suspect the transaction is problematic.
6. `process_transaction` iterates all logs. Both transfer events match `ERC20::Transfer::SIGNATURE_HASH`. Both are processed as deposits:
7. First: `process_deposit(from=User, value=100)` — mints 100 receipt tokens. Correct.
8. Second: `process_deposit(from=User, value=50,000)` — mints 50,000 receipt tokens.
No tokens entered escrow for this one.
9. The user now holds 50,100 receipt tokens on L2 but only deposited 100 USDC into escrow. The user withdraws the full 50,100, draining the escrow of other depositors' funds.
10. There is no administrative function to reverse the phantom deposit. The escrow is now undercollateralized by 50,000 USDC.

This attack passes through an honest admin because the transaction genuinely contains a valid escrow deposit; the admin has no mechanism to selectively process only certain Transfer events from the transaction.

Recommendation

Register escrow addresses per chain at the module level and validate that the `to` field of every `ERC20::Transfer` event matches the expected escrow address:

```
ERC20::Transfer::SIGNATURE_HASH => {  
  let ERC20::Transfer { from, to, value } =  
    ERC20::Transfer::decode_raw_log(topics, log.data.as_ref());  
  
  let expected_escrow = self.get_escrow_address(chain_hash.chain_id, state)?;  
  if to != expected_escrow {  
    continue; // skip non-escrow transfers  
  }  
}
```



```
let e = self.process_deposit(  
  S::Address::try_from(from.as_ref())?,  
  ChainAddress::try_from_chain_id_and_data_bytes(  
    chain_hash.chain_id,  
    log.address.as_ref(),  
  )?,  
  value.try_into()?,  
  state,  
)?;  
// ...  
}
```

Additionally, the transaction-level `from` and `to` fields in `TxVerificationData::Evm` are currently discarded (`from: _, to: _`). Once per-chain escrow address registration is implemented, cross-checking `tx_data.to` against the registered escrow address provides an additional defense-in-depth layer beyond the log-level `Transfer.to` check.

Fix Implemented

The fix registers expected escrow contract addresses per chain in the config, then validates `log.address` against those registered addresses in `process_transaction` — `Transfer` events from addresses not in `chain_config.escrow_contracts` are now skipped, preventing phantom deposits from arbitrary ERC-20 emitters.



Medium findings

[M-01] Withdrawal confirmation burns amounts without validating against stored request

Severity

Impact: Medium

Likelihood: Medium

Description

When a withdrawal is created in `execute_intent_auth()`, the escrow ledger is decremented and receipt tokens are transferred based on the user's signed intent data at

`execute_intent_auth.rs:79-114` :

```
// execute_intent_auth.rs:79-85
for transfer in &decoded.request.transfers {
    let chain_token = ChainAddress::try_from_chain_id_and_data(chain, &transfer.token)?;
    let raw_amount = transfer.amount.try_into()?;
    let amount = Amount(raw_amount);

    // update ledger
    self.modify_escrow_balance(&chain_token, |b| b.checked_sub(amount), state)?;
```

- These amounts are stored in the `WithdrawalRequest` at `execute_intent_auth.rs:92-97` :

```
// execute_intent_auth.rs:92-97
transfers.push(WithdrawalRequestTransfer {
    token_in: receipt_token,
    token_out: transfer.token.clone(),
    recipient: transfer.to.clone(),
    amount: raw_amount,
});
```

- When the withdrawal is later confirmed via `process_withdrawal_confirmed()`, the stored request is fetched at `process_transaction.rs:173-179`, but only `request.chain` and `request.state` are read from it. The `burn` amounts, `token addresses`, and `transfer count` are taken entirely from the on-chain `WithdrawalProcessed` event at `process_transaction.rs:206-211` :

```
// process_transaction.rs:206-211
for transfer in withdrawal_processed.request.transfers {
    let chain_token = ChainAddress::try_from_chain_id_and_data_bytes(
        request.chain,
```



```
transfer.token.as_ref(),
)?;
let amount = Amount(transfer.amount.try_into()?);
```

The stored `request.transfers` is never compared against the `event's` transfers. The only link between creation and confirmation is the `idempotenceKey`. No validation occurs that:

- The number of transfers in the event matches the stored request.
- The token addresses match.
- The amounts match.

This creates two concrete scenarios where the escrow accounting invariant breaks permanently.

- Scenario 1: Fee-on-transfer tokens cause amount mismatch

The `L1 escrow contract` executes a `withdrawal` by calling `transfer()` on the underlying ERC20 token. When the token charges a fee on transfer, the actual amount received by the `recipient` is less than the amount specified in the `WithdrawalProcessed` event. However, the event logs the pre-fee amount.

- The [USDT contract on Ethereum](#) has a dormant fee mechanism that Tether can enable at any time:

```
uint public basisPointsRate = 0; // currently zero, can be set up to 20 (0.2%)
uint public maximumFee = 0;

function setParams(uint newBasisPoints, uint newMaxFee) public onlyOwner {
    require(newBasisPoints < 20);
    require(newMaxFee < 50);
    basisPointsRate = newBasisPoints;
    maximumFee = newMaxFee.mul(10**decimals);
}
```

- Tokens that currently charge transfer fees include `STA` and `PAXG` ([source](#)).

In the Spicenet context:

- User creates withdrawal for `1000 USDT`. Escrow is decremented by `1000`. 1000 receipt tokens are transferred from user to admin.
- On L1, escrow sends 1000 USDT, but USDT deducts a 0.2% fee. Recipient gets 998 USDT.
- `WithdrawalProcessed` event logs amount = 1000 (pre-fee value).
- Admin submits this `event.process_withdrawal_confirmed`, which burns 1000 receipt tokens from admin.
- Accounting appears balanced on the rollup, but L1 only delivered 998. The 2 USDT fee is absorbed silently.
- Over many withdrawals, the cumulative gap grows, and the escrow ledger overstates actual L1 holdings.



Alternatively, if the event were to log the post-fee amount (998), then only 998 receipt tokens would be burned, leaving 2 unbacked receipt tokens with the admin. Either way, accounting breaks.

- Scenario 2: Partial multi-transfer execution creates permanent token loss

A withdrawal request can contain multiple transfers (`CreateTokenTransfer[]`). On L1, if one transfer in the batch reverts (e.g., recipient contract rejects the token, the token is paused, insufficient liquidity), the escrow contract processes only the successful transfers.

- User creates withdrawal with 3 transfers: `100 USDC, 50 WETH, 200 DAI`. Escrow is decremented for all three. All three receipt token types are transferred from user to admin.
- On L1, the DAI transfer reverts. Escrow emits `WithdrawalProcessed` with only 2 transfers.
- Admin submits the 2-transfer event to `process_transaction`.
- `process_withdrawal_confirmed` at line 206 iterates only 2 transfers. Only USDC and WETH receipt tokens are burned.
- At line 185, the entire request is marked as successful.
- 200 DAI receipt tokens remain with admin, never burned. Escrow was decremented by 200 DAI at creation. No mechanism exists to revisit or refund. The gap is permanent because `WithdrawalRequestState::Failed` is never set by any code path and no partial-failure handling exists.

There is an additional ordering issue: in `process_withdrawal_confirmed`, the request is marked `Successful` and removed from `withdrawal_request_ids_active` before the token burn loop begins executing. If any individual burn within the loop fails (for example, because the admin's balance of a specific receipt token is exhausted), the request has already been permanently committed as confirmed with no mechanism to retry or reverse.

Accounting invariant proof

Let $E_d = \text{total escrow decrement at creation} = \sum(\text{stored_amounts})$. Let $B_c = \text{total burns at confirmation} = \sum(\text{event_amounts})$. The accounting invariant requires $E_d == B_c$. Since these values are sourced independently — E_d from the user's signed intent, B_c from the admin-submitted L1 event, and no cross-validation enforces equality, the invariant can be violated by $|E_d - B_c|$ on every confirmation. The gap is cumulative and irreversible.

Recommendations

Implement cross-validation in `process_withdrawal_confirmed` before burning:

- Assert that `withdrawal_processed.request.transfers.len() == request.transfers.len()`.
- For each transfer, assert that the event's token address and amount match the stored request's `token_out` and `amount`.



Fix Implemented

`process_withdrawal_confirmed` now cross-validates the on-chain `WithdrawalProcessed` event against the stored `WithdrawalRequest`: it checks that transfer counts match and that each transfer's `recipient` and `token_out` fields (now typed as `ChainAddress` instead of `String`) are identical, returning `TokenTransferCountMismatch` or `TokenTransfersMismatch` errors on any discrepancy.

[M-02] Withdrawal confirmation lacks validation of `chain_hash` against stored request

Severity

Impact: High

Likelihood: Low

Description

When processing a `WithdrawalProcessed` event, the module never checks that the event came from the same chain for which the withdrawal was created. `chain_hash` — which carries the submitting chain's ID — is available in `process_transaction` but is never forwarded to `process_withdrawal_confirmed`:

```
// process_transaction.rs
WithdrawalProcessed::SIGNATURE_HASH => {
    let withdrawal_processed =
        WithdrawalProcessed::decode_raw_log(topics, log.data.as_ref())?;
    // chain_hash is in scope here but never passed down
    let e = self.process_withdrawal_confirmed(withdrawal_processed, state)?;
}
```

Inside `process_withdrawal_confirmed`, the only chain context used is `request.chain` — the chain the withdrawal was supposed to target:

```
// process_withdrawal_confirmed
let Some(mut request) = self.withdrawal_requests.get(&idempotence_key, state)?
    else { return Err(...) };

request.validate_state(WithdrawalRequestState::Created)?;
request.state = WithdrawalRequestState::Successful;
```

A `WithdrawalProcessed` event extracted from a transaction on chain B can successfully confirm a withdrawal request that was created for chain A. The idempotence key is the only thing checked — not the originating chain.



Scenario:

1. User creates a withdrawal request targeting Ethereum mainnet (`request.chain = evm:1`).
2. The Ethereum escrow never executes the withdrawal.
3. Admin submits a `WithdrawalProcessed` event embedded in a BSC transaction (`chain_hash.chain_id = evm:56`).
4. The event's `idempotence_key` matches the stored request.
5. `process_withdrawal_confirmed` marks the request `Successful` and burns the admin's receipt tokens.
6. The user never receives funds on Ethereum — the withdrawal is permanently marked confirmed with no recourse.

Recommendations

Consider passing `chain_hash.chain_id` into `process_withdrawal_confirmed` and assert that it matches `request.chain` immediately after the request is loaded:

```
if chain_hash.chain_id != request.chain {  
    return Err(SettlementsError::ChainMismatch { ... });  
}
```

Fix Implemented

The `process_withdrawal_confirmed` function now receives the `ChainTransaction` context and compares `request.chain` against `chain_transaction.chain_id`, returning `SettlementsError::InvalidChainId` if they differ, ensuring a withdrawal confirmed on the wrong chain is rejected.

[M-03] Failed withdrawals have no recovery path permanently locking user values

Severity

Impact: Medium

Likelihood: Medium

Description

When a withdrawal is created in `execute_intent_auth()`, the user's receipt tokens are immediately transferred to the admin at `execute_intent_auth.rs:100-114`:

```
// execute_intent_auth.rs:100-108  
self.sov_bank  
    .transfer_from(  
        <...>
```




```
&user,  
&admin,  
Coins {  
    token_id: receipt_token,  
    amount,  
},  
state,  
)
```

And the escrow ledger is decremented at line 85:

```
// execute_intent_auth.rs:85  
self.modify_escrow_balance(&chain_token, |b| b.checked_sub(amount), state)?;
```

The request is persisted with `WithdrawalRequestState::Created` at `execute_intent_auth.rs:124`. The only state transition implemented is `Created -> Successful` in `process_withdrawal_confirmed()` at `process_transaction.rs:182-185`:

```
// process_transaction.rs:182-185  
request.validate_state(WithdrawalRequestState::Created)?;  
request.state = WithdrawalRequestState::Successful;
```

While `WithdrawalRequestState::Failed` exists as an `enum variant` (`withdrawal_request.rs:44`), no code path in scope ever transitions a request to `Failed`, refunds the receipt tokens from admin back to the user, or restores the escrow ledger balance. If a withdrawal fails on the destination chain, the user's tokens remain locked in admin custody forever with no mechanism for recovery.

Recommendations

Implement an explicit failure/cancellation flow that:

- Transitions the request state to `Failed`.
- Restores the `spice_escrow_ledger` balance via `checked_add`.
- Transfers the receipt tokens from admin back to the user.
- Removes the request from `withdrawal_request_ids_active`.
- Consider also allowing users to self-cancel after the withdrawal's `exp` timestamp has elapsed: if the current block time exceeds `exp` and the request is still in `Created` state, the user should be able to trigger the refund directly without requiring admin action, reducing trust assumptions on the admin's availability.

Fix Implemented

A new `CancelWithdrawalRequest` admin call is added that transitions a `Created` withdrawal request to `Canceled` state within a configurable cooldown window. The cancellation handler refunds the user's receipt tokens using `normalize_decimals` for decimal-correct amounts, and restores the escrow ledger balance, ensuring no user funds remain locked on failed L1 withdrawals.



Low findings

[L-01] Hardcoded user address in `ExecuteIntentAuth` restricts chain-agnostic design

Description

The `ExecuteIntentAuth` call message and its supporting intent infrastructure hardcode user addresses to 20 bytes (`HexString<[u8; 20]>`), which correspond to the EVM address format. This constraint is present in:

1. `CallMessage::ExecuteIntentAuth` in `crates/settlements/src/call/mod.rs` — the `user` field is typed as `HexString<[u8; 20]>`.
2. `Settlements::execute_intent_auth()` in `crates/settlements/src/call/execute_intent_auth.rs` — the function parameter mirrors the same 20-byte constraint.
3. `IntentCall.to` in `crates/shared/src/intent_wire.rs` — the call target address is also `HexString<[u8; 20]>`.

Additionally, `compute_chain_batch_hash_from_wire()` in `crates/shared/src/intent.rs` converts `IntentCall.to` into `AbiAddress` (an EVM address type) and encodes it via `abi_encode_params`, further coupling the intent system to EVM semantics.

This is inconsistent with the rest of the system, which is designed to be chain-agnostic:

- `ChainId` explicitly supports `Svm { network }` for Solana chains.
- `ChainAddress` handles 32-byte Solana addresses via base58 encoding.
- `WalletType` supports `Solana { address: [u8; 32] }`, `Aptos { address: [u8; 32] }`, and `Sui { address: [u8; 32] }`.
- The Capsule module correctly verifies Ed25519 signatures for Solana wallets.

Recommendation

Replace the fixed 20-byte user address with a variable-length representation that can accommodate addresses from all supported chains:

In `crates/settlements/src/call/mod.rs`:

```
pub enum CallMessage {
    ExecuteIntentAuth {
        user: HexString<Vec<u8>>, // accepts 20-byte EVM and 32-byte Solana/Aptos/Sui
        intent_auth: IntentAuth,
    },
}
```

In `crates/settlements/src/call/execute_intent_auth.rs`:



```
pub fn execute_intent_auth(
    &mut self,
    user: HexString<Vec<u8>>,
    intent_auth: IntentAuth,
    context: &Context<S>,
    state: &mut impl TxState<S>,
) -> Result<(), SettlementsError> {
    let user = S::Address::try_from(user.0.as_slice()).map_err(|e|
SettlementsError::InvalidAddress {
    message: e.to_string(),
    address: hex::encode(&user.0),
    })?;
    // ...
}
```

[L-02] Withdrawal requests persist unbounded user-controlled strings

Description

The module stores several user-controlled strings directly in persistent withdrawal state, including `metadata`, `token_out`, and `recipient`, without any explicit length checks. Although transaction-size limits provide some upper bound, there is no application-level constraint preventing users from maximizing these fields and inflating stored state.

```
let request = WithdrawalRequest {
    state: WithdrawalRequestState::Created,
    chain,
    idempotence_key,
    for_user: user,
    nbf: decoded.request.nbf.try_into()?,
    exp: decoded.request.exp.try_into()?,
    metadata: decoded.request.metadata,
    requested_at: now,
    transfers,
};
```

```
pub nbf: u64,
pub exp: u64,
pub metadata: String,
pub requested_at: u64,

pub transfers: Vec<WithdrawalRequestTransfer>,
```

```
pub struct WithdrawalRequestTransfer {
    pub token_in: TokenId,
    pub token_out: String,
    pub recipient: String,
    pub amount: u128,
}
```



This is primarily a state-bloat concern rather than a direct safety issue, but it increases long-term storage pressure and compounds the impact of unbounded request creation patterns.

Recommendation:

add explicit maximum lengths for `metadata`, `token_out`, and `recipient`, and reject requests that exceed those caps before writing them to state. A concrete starting bound for the `metadata` field is 1024 bytes (e.g., `const MAX_METADATA_LEN: usize = 1024; if decoded.request.metadata.len() > MAX_METADATA_LEN { return Err(SettlementsError::MetadataTooLarge); }`). Smaller limits should be applied to `token_out` and `recipient` based on the maximum address and identifier lengths of supported destination chains.

Fix Implemented

A 1024-byte cap is enforced on `WithdrawalRequest.metadata` in `create_withdrawal_request`: if `metadata.len() > 1024`, the call returns `SettlementsError::WithdrawalRequestMetadataTooLong`, preventing unbounded state growth from user-controlled string fields.

[L-03] Zero-amount deposits and withdrawals are accepted

Description

The settlements flow accepts `amount == 0` for both deposits and withdrawals. In `process_deposit()`, a zero-value ERC20 Transfer log is still processed into `Amount(0)`, updates the escrow ledger with a no-op, and proceeds to mint via the bank module. In `execute_intent_auth()`, a zero-amount withdrawal similarly passes through ledger mutation, receipt-token transfer, and full withdrawal request creation.

```
let amount = Amount(amount_raw);
self.modify_escrow_balance(&chain_token, |b| b.checked_add(amount), state)?;

let coins = Coins {
    amount,
    token_id: receipt_token,
};

self.sov_bank
    .mint(coins, &user, &admin, state)
```

```
for transfer in &decoded.request.transfers {
    let chain_token = ChainAddress::try_from_chain_id_and_data(chain, &transfer.token)?;
    let raw_amount = transfer.amount.try_into()?;
    let amount = Amount(raw_amount);

    self.modify_escrow_balance(&chain_token, |b| b.checked_sub(amount), state)?;
    // ...
```



```
transfers.push(WithdrawalRequestTransfer {
    token_in: receipt_token,
    token_out: transfer.token.clone(),
    recipient: transfer.to.clone(),
    amount: raw_amount,
});
}
```

While this does not directly lose funds, it allows useless zero-value operations to consume persistent state, including withdrawal request storage, the active request set, and per-user request indexing. This contributes to avoidable state growth and operational noise.

Recommendation:

- Reject zero-amount deposits and withdrawals early with an explicit `amount > 0` check before mutating state or creating request records.

For `execute_intent_auth`, add the following early check at the start of the transfer loop before any state mutation: `let raw_amount: u128 = transfer.amount.try_into()?; if raw_amount == 0 { return Err(SettlementsError::InvalidAction); }` — this rejects zero-amount withdrawals before escrow is decremented or receipt tokens are transferred.

Client Commentary

Zero value transfers are valid operations on most tokens. I feel it makes more sense to allow them so the rollup can more accurately reflect the on-chain actions.

[L-04] Only the first call in a signed batch is executed while others are discarded

Description

`execute_intent_auth()` authenticates the user's signature over an entire batch of calls, but only executes the first one. The remaining calls are silently discarded with no error, event, or revert.

- The flow:
- **Line 26-30** — Finds the first `chain_batch` with `non-empty` calls. All other batches are dropped:

```
// execute_intent_auth.rs:26-30
let active_batch = intent_auth
    .chain_batches
    .iter()
    .find(|b| !b.calls.is_empty())
    .ok_or(SettlementsError::InvalidAction)?;
```

1. **Line 33-34** — Computes a hash over the entire active batch, including all calls:



```
// execute_intent_auth.rs:33-34
let active_hash =
    spicenet_shared::intent::compute_chain_batch_hash_from_wire(active_batch).into();
```

1. `Line 43-48` — `verify_intent()` authenticates the user's signature against that full batch hash; the user signed for all calls in the batch:

```
// execute_intent_auth.rs:43-48
self.capsule
    .verify_intent(&user, &intent_auth, active_hash, &required_scopes, state)
    .map_err(|e| SettlementsError::InvalidIntentAuth {
        message: e.to_string(),
        intent_auth: intent_auth.clone(),
    })?;
```

1. Only `.first()` is extracted. All subsequent calls are silently ignored:

```
// execute_intent_auth.rs:51-54
let call = active_batch
    .calls
    .first()
    .ok_or(SettlementsError::InvalidAction)?;
```

1. Only that single call is decoded and executed. The transaction succeeds. The TODO at line 23 confirms this is known incomplete behavior:

```
// execute_intent_auth.rs:23
// TODO: refactor to process all chain batches in loop
```

A user who signs a batch containing N withdrawal requests will have only `calls[0]` executed. Its escrow is decremented, its receipt tokens transferred to admin, and its `WithdrawalRequest` persisted. Withdrawals `calls[1..N]` are silently dropped with no state change and no event emitted. The user cannot resubmit the same `intent_auth` to recover the remaining calls because `calls[0]`'s `idempotence_key` already exists and would be rejected at `execute_intent_auth.rs:66-73`.

```
// execute_intent_auth.rs:66-73
if self
    .withdrawal_requests
    .get(&idempotence_key, state)
    .map_err(SettlementsError::state_access_error)?
    .is_some()
{
    return Err(SettlementsError::WithdrawRequestAlreadyExists { idempotence_key });
}
```

Additionally, the entire intent's nullifier is consumed via `processed_intent_steps` when any call from the intent is processed. This means the user cannot resubmit the same signed intent to recover the silently dropped calls — the full authorization covering all calls is irreversibly burned after only the first call executes. The user must sign an entirely new intent to reauthorize the dropped withdrawal calls, and there is no indication from the original transaction that any calls were skipped.



recommendation

Implement the TODO and iterate over all calls in the active batch, decode and execute each one independently. Each call should have its own `idempotence_key` check and its own state transition.

Fix Implemented

`execute_intent_auth` is refactored to iterate over all chain batches and all calls within each batch (dispatching by 4-byte sighash), replacing the previous logic that only executed the first call of the first non-empty batch. The withdrawal creation logic is extracted into `create_withdrawal_request()`, and the intent nullifier now covers each (intent_hash, chain_batch_hash) pair individually.

[L-05] Admin address is immutable no key rotation mechanism

Description

The admin address is set once at genesis (`lib.rs`) via `SettlementsConfig.sequencer_authority` and cannot be changed after deployment. Every `CallMessage` in the settlements module (`UpdateMapping` , `ProcessTransaction` , `ExecuteIntentAuth`) is gated by `verify_admin()`, making the admin the single point of control for all bridge operations.

There is no `set_admin`, multisig, timelock, or emergency transfer function anywhere in the module.

Recommendations:

Consider implementing an admin transfer mechanism, at a minimum, a two-step `propose_admin` / `accept_admin` pattern to prevent lockout.

Fix Implemented

Deposit processing in `process_transaction` now validates that the `Transfer` event's emitting contract address matches a registered escrow address for the given chain — events from unregistered contracts are silently skipped, closing the phantom deposit vector.

[L-06] Inconsistent `HashSet` implementations across the crate

Description

`lib.rs` imports `ahash::HashSet` for `withdrawal_request_ids_active`, while `state/mod.rs` uses `std::collections::HashSet` for `TokenMapConfigType`. The two implementations use different hashing algorithms.



Standardize on one `HashSet` implementation across the crate.

Fix Implemented

The `ahash::HashSet` import in `settlements/src/lib.rs` is replaced with `std::collections::HashSet`, ensuring consistent behavior and eliminating the dependency on a non-standard hash set implementation.

[L-07] Smart contract wallets unable to withdraw funds from L1 escrow

Description

When a smart contract wallet (e.g., Gnosis Safe, multisig, or any contract account) deposits tokens into the L1 escrow, the `process_deposit` flow mints receipt tokens to `S::Address(from)` where `from` is the smart contract's address. However, the only withdrawal path requires `execute_intent_auth`, which calls `capsule.verify_intent()`. Inside `verify_intent`, the signature is verified via ECDSA recovery:

```
let recovered = recover_signer(digest, intent.signature.as_ref())?;
```

Smart contract wallets do not have private keys and cannot produce ECDSA signatures. There is no EIP-1271 (`isValidSignature`) support in the capsule's `verify_intent`; it exclusively uses `ecrecover`. This means:

1. A smart contract wallet deposits to the L1 escrow.
2. Receipt tokens are minted to the contract's address on Spicenet.
3. The contract cannot sign an `IntentAuth` because it has no private key.
4. No withdrawal request can ever be created for those funds.
5. The funds are permanently locked with no recovery mechanism.

This is a known class of issues in L2/bridge systems. For example, Optimism's early bridge design assumed L1 and L2 addresses were equivalent, which caused problems for smart contract wallets whose L2 counterpart was either a different contract or an EOA controlled by someone else. Several users lost funds bridging from multisigs before mitigations were added. The same category of risk applies here; any non-EOA depositor is silently accepted but permanently unable to withdraw.

Recommendation

Either reject deposits from smart contract wallets by verifying that the `from` address has associated EOA characteristics, or add EIP-1271 signature verification support in `verify_intent` to allow smart contract wallets to authorize withdrawals:



```
// In verify_intent, after ecrecover fails:
// Fall back to EIP-1271 for contract wallets
let is_valid = self.call_is_valid_signature(capsule_address, digest, signature)?;
if !is_valid {
    bail!("signature verification failed");
}
```

Alternatively, allow the admin to process refund deposits back to L1 for addresses that cannot create capsules.

Fix Implemented

The intent verification layer is refactored into `crates/shared/src/intent/` and extended with EIP-1271 smart wallet support: `verify_signature()` now accepts an optional `eip_1271_response` parameter and handles the `"eip-1271"` signature type by checking the magic value `0x1626ba7e`, allowing contract-based wallets to authorize withdrawals.

[L-08] Inadequate validation in `update_mapping` function for token removal

Description

The `update_mapping` function in `crates/settlements/src/call/update_mapping.rs` performs a full replacement of the `token_map_config` state without validating whether any users hold non-zero receipt token balances or have active (state: `Created`) withdrawal requests referencing tokens being removed from the mapping.

The only validation performed is that each token in the `new` mapping exists in `sov_bank`. There is no comparison between the old and new mappings, no check for outstanding supply of receipt tokens being de-listed, and no check against the `withdrawal_request_ids_active` set.

When a chain token entry is removed from the mapping, `get_receipt_token` (in `crates/settlements/src/get.rs`) returns `None` for the removed token. This causes `ReceiptTokenNotFoundForChainToken` errors in three critical paths:

1. `execute_intent_auth` — Users cannot create withdrawal requests to reclaim their receipt tokens.
2. `process_withdrawal_confirmed` — Pending withdrawal requests created before the mapping removal cannot be finalized. The confirmation reverts, leaving the user's receipt tokens locked in the administrator's custody with no path to completion.

This vulnerability involves a time-of-check-to-time-of-use gap: the receipt token for a withdrawal is resolved via `get_receipt_token` at confirmation time inside `process_withdrawal_confirmed` (approximately `process_transaction.rs:213-215`),



not at withdrawal creation time. A mapping change between creation and confirmation silently changes which receipt token is resolved — or resolves to `None` entirely — at the moment the burn is attempted.

Attack Flow Example

1. Admin configures the token mapping: `{weth_receipt_token => [eth_mainnet:0xC02...WETH]}`. Users deposit WETH on L1 and receive `weth_receipt_token` on the rollup.
2. Alice deposits 10 WETH into the L1 escrow. The admin calls `process_transaction` — Alice receives 10 `weth_receipt_token`.
3. Alice signs an intent to withdraw 5 WETH. The admin calls `execute_intent_auth`, which transfers 5 `weth_receipt_token` from Alice to the admin and creates a `WithdrawalRequest` with state `Created`.
4. Before Alice's withdrawal is confirmed on L1, the admin calls `update_mapping` with a new mapping that omits the WETH entry (e.g., during a routine config update that adds a new token but accidentally drops the existing one). The call succeeds with no warnings.
5. Alice's withdrawal is confirmed on L1 and emits `WithdrawalProcessed`. The admin submits the proof, but `process_withdrawal_confirmed` attempts `get_receipt_token` for WETH, receives `None`, and `reverts`. The 5 `weth_receipt_token` held by the admin cannot be burned.
6. Alice still holds 5 `weth_receipt_token` but cannot create a new withdrawal request because `execute_intent_auth` also fails at the `get_receipt_token` lookup. Her remaining receipt tokens are stranded.

Recommendation

Add safety checks to `update_mapping` that prevent the removal of token entries when there are outstanding escrow balances or active withdrawal requests:

```
pub fn update_mapping(
    &mut self,
    new_mapping: TokenMapConfigType,
    context: &Context<S>,
    state: &mut impl TxState<S>,
) -> Result<(), SettlementsError> {
    let caller = self.verify_admin(context, state)?;

    // ... existing token existence validation ...

    // Collect all chain tokens in the new mapping
    let new_chain_tokens: HashSet<&ChainAddress> = new_mapping
        .values()
        .flat_map(|tokens| tokens.iter())
        .collect();

    // Check that removed chain tokens have zero escrow balance
    let old_mapping = self.get_token_map_config(state)?;
    for chain_token in old_mapping.values().flat_map(|tokens| tokens.iter()) {
        if new_chain_tokens.contains(chain_token) {
```



```
        continue;
    }

    let balance = self.spice_escrow_ledger
        .get(chain_token, state)
        .map_err(SettlementsError::state_access_error)?
        .unwrap_or(Amount(0));
    if balance.0 > 0 {
        return Err(SettlementsError::CannotRemoveTokenWithBalance {
            chain_token: chain_token.clone(),
            balance: balance.0,
        });
    }
}

self.token_map_config.set(&new_mapping, state)
    .map_err(SettlementsError::state_access_error)?;
// ...
}
```

Alternatively, transition `update_mapping` to an additive or merge-only strategy: disallow removal of any `ChainAddress` entry that has a non-zero `spice_escrow_ledger` balance or any active entry in `withdrawal_request_ids_active`, allowing additions and modifications only. This avoids the need to enumerate all active requests at update time.

[L-09] Per-user withdrawal history grows unbounded with $O(n)$ rewrite cost per request

Description

In `execute_intent_auth.rs:144-152`, every new withdrawal request appends to a per-user `Vec<HexHash>` that is loaded in full, extended, and rewritten:

```
// execute_intent_auth.rs:144-152
let mut user_requests = self
    .withdrawal_request_ids_by_user
    .get(&request.for_user, state)
    .map_err(SettlementsError::state_access_error)?
    .unwrap_or_default();
user_requests.push(request.idempotence_key);
self.withdrawal_request_ids_by_user
    .set(&request.for_user, &user_requests, state)
```

The field is declared as `StateMap<S::Address, Vec<HexHash>>` at `lib.rs:77`. This vector is never pruned; `completed` and `failed` requests remain in the list forever. For a user with `n` historical withdrawals, every new withdrawal requires deserializing, cloning, and re-serializing the full vector. At scale, this causes $O(n)$ gas cost growth per withdrawal per user, eventually making withdrawals prohibitively expensive or causing transaction timeouts for active users.



Recommendations

Replace `StateMap<S::Address>`, `Vec<HexHash>>` with an indexed or append-only structure (e.g., a per-user `StateVec`) that supports $O(1)$ appends without full deserialization. If historical lookup is needed, separate active requests from completed ones.

[L-10] No validation of `log.address` in `WithdrawalProcessed` event enables attack

Description:

The `WithdrawalProcessed` event handler at `process_transaction.rs:83-94` decodes the event payload from `log.topics` and `log.data`, but never inspects or validates `log.address`, the address of the contract that emitted the event.

```
WithdrawalProcessed::SIGNATURE_HASH => {
    let withdrawal_processed =
        WithdrawalProcessed::decode_raw_log(topics,
log.data.as_ref())
        .map_err(|e| SettlementsError::CouldNotDecodeLog {
            transaction: chain_hash,
            message: e.to_string(),
        })?;
    let e =
        self.process_withdrawal_confirmed(withdrawal_processed,
state)?;
    self.emit_event(state, e.clone());
    events.push(e);
}
```

In contrast, the ERC20 Transfer handler does use `log.address` at line 73 to construct the `chain_token`, providing indirect validation through the token mapping lookup. Any contract on the source L1 can emit an event with the same full 32-byte event signature hash as `WithdrawalProcessed`.

The core problem is that `WithdrawalProcessed` is accepted based only on `topic0` plus the decoded payload. The handler never validates `log.address`, never checks `tx_data.to`, and never verifies that the transaction's chain matches the withdrawal's stored chain.

- There is no in-scope enforcement that the admin only submits logs from a registered escrow, and the code even contains a TODO for escrow address validation on deposits.



Impact:

- When the admin relay faithfully relays all events from an L1 transaction receipt, a rogue contract's `WithdrawalProcessed` event is processed as a legitimate withdrawal confirmation. This:
- Sets a pending `withdrawal` to `Successful`, permanently blocking the real confirmation (`validate_state(Created)` rejects it)
- Burns arbitrary amounts of arbitrary receipt tokens from the admin based on the attacker-controlled transfers array.
- Permanently blocks the legitimate escrow confirmation because `validate_state(Created)` will reject any later real event.

Attack Scenario:

- Victim creates a withdrawal for `1000 receipt_USDC`. The `idempotence_key` is visible in the emitted `WithdrawalCreated` event.
- Attacker deploys an L1 contract that emits `WithdrawalProcessed({idempotenceKey: <victim's key>, transfers: []})` (empty transfers = zero burns).
- Admin relay processes this transaction. The withdrawal is marked successful with zero tokens burned.
- The real L1 confirmation is rejected. `1000 receipt_USDC` is orphaned with the admin.

Recommendation:

Store a mapping of `ChainId -> escrow_contract_address` in module state. Validate `log.address` matches the registered escrow address before processing any `WithdrawalProcessed` event.

Fix Implemented

`WithdrawalProcessed` events are now validated against the registered escrow contract address: `log.address` is resolved to a `ChainAddress` and checked against `chain_config.escrow_contracts`, skipping any events not originating from a known escrow contract.